

Plug&Pwn: Weaponizing Windows PnP

Alejandro Hernando @0xedh & Borja Martínez @borjnz

0xedh@defcon34:~\$ whoami

```
0xedh@defcon34
```

```
// Alejandro Hernando / @0xedh  
//  
// Red team and researcher in the Hacking Accenture  
// Spain team.  
//  
// I like to break things in my free time  
// and spend money on gadgets.  
//  
// Github: @0xedh  
// Telegram: @edhx0  
// Twitter: @0xedh
```



borjnz@defcon34:~\$ whoami

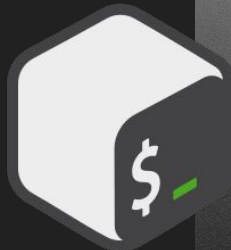
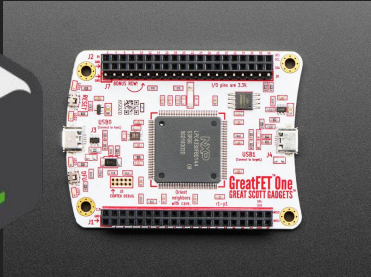
```
borjnz@defcon34
```

```
// Borja Martinez / @borjnz  
//  
// Red team and researcher in the Hacking Accenture  
// Spain team.  
//  
// From time to time I play the occasional CTF  
// and was part of the ID-10-T (Retired) team.  
//  
// Telegram: @borjnz  
// Twitter: @Qm9yamFN  
// Github: @borjnz
```



“Physical” access -> **SYSTEM**. No clicks.

What you're about to watch, side by side:

Left - the target Windows 11, fully updated	Right - us Linux + a FaceDancer
 Windows 11 No user logged on. Nothing pre-installed. A clean, patched machine.	  Our exploit chain: emulate two USB devices, hijack DNS through PIPE, plant a DLL, load it as SYSTEM.

POC_0 - First exploit chain

Why a pile of "minor" bugs ends in SYSTEM

- We chain low-severity bugs, across vendors, into one critical.
- On their own they're minor. Some vendors won't even call them vulnerabilities. Chained the right way, together they hand us arbitrary code execution as SYSTEM.

1 - Sierra Wireless	2 - Sony FeliCa	3 - Then ...
EM7340 modem utility SwiService.exe runs as SYSTEM	NFC reader co-installer pulls its files over HTTP	Doing it remotely. No hardware.

Sierra Wireless “SwiService.exe (1/2)”

```
43 local_b0 = GetTickCount64();
44 if (param_1[3] == 0) {
45     BVar3 = InitializeSecurityDescriptor(local_90,1);
46     if (BVar3 != 0) {
47         pwVar9 = (wchar_t *)0x0;
48         /* NULL DACL, no access control */
49         BVar3 = SetSecurityDescriptorDacl(local_90,1,(PACL)0x0,0);
50         if (BVar3 != 0) {
51             local_d0.nLength = 0x18;
52             local_d0.lpSecurityDescriptor = local_90;

// Exposes a named pipe with an Everyone r/w ACL.
```

```
pvVar12 = CreateNamedPipeW(param_2,0x40000003,6,0xff,param_4,0,5000,
                           &local_d0);
if (pvVar12 == (HANDLE)0xffffffffffffffff) {
    GetLastError();
    pwVar9 = L"Failed to create named pipe [%s]. Err - %d";
    lpHandles = (HANDLE *)0x3;
    FUN_140087070(6,3,L"Failed to create named pipe [%s]. Err - %d",param_2);
}

// Any local user or any domain user can connect and
// call SetDNS function remotely.
```

Sierra Wireless “SwiService.exe (2/2)”

```
// The executable is hardcoded to netsh.  
// The interface name comes from a system lookup.  
  
89     local_12d0 = local_10b8;  
90     local_12d8 = *(undefined4 **)((longlong)pvVar2 + 0x48);  
91     FUN_140014940(local_838,0x400,L"interface ipv%d add dns \"%s\" %s",(ulonglong)uVar8);  
92     FUN_1400027c0((undefined (*) [16])L"netsh.exe",local_838,0,1);  
93 }  
94 else {  
95     if ((bool)*(char *)(param_1 + 1) != (iVar3 == 0)) {  
96         uVar8 = uVar9;  
97     }  
98     FUN_140087070(6,3,L"Error retrieving IPv%d address string",(ulonglong)uVar8);  
99 }  
100 GlobalFree(local_12b0);  
  
// The intended effect, point DNS at any IP (under our control).
```

SONY + SIERRA - Sierra recap

1 emulate Sierra EM7340 1199:A000

SIERRA

SmiService.exe (SYSTEM) opens a NULL-DACL pipe \\.\pipe\SmiServicePipe -
Everyone R/W, local or over SMB/445



2 call SetDns (msg type 9) -> hijack DNS

SIERRA

service runs netsh ... add dns as SYSTEM -> machine DNS = attacker;
FlushDNS (type 8) for instant effect



3 attacker serves DNS

www.sony.co.jp -> attacker IP; everything else -> 8.8.8.8

Sony Felica “felica_coinst.dll (1/2)”

```
19             /* 0x7334 1 ClassInsatller
20             0x7334 2 ClassInstaller
21 This is not my typo! Sony's own typo in the export table. */
22 local_d8 = 0xfffffffffffffffe;
23 local_28 = DAT_0202a1b8 ^ (ulonglong)auStackY_138;
24 local_e8 = (longlong *)FUN_020077b8();
25 local_e0 = 0;
26 FUN_02006c90();
27 DAT_0202bfc0 = param_2;
28 DAT_0202bfc8 = param_3;
29 if (param_1 == 0x1e) {
30             /* Runs the orchestrator as SYSTEM. It downloads config
31             files and writes them to disk.
32             This is where path traversal triggers */
33     puVar4 = (undefined *)ORCHESTRATOR_DOWNLOAD();
```

```
142     iVar2 = FUN_02013a90((longlong)&local_548);
143     if (iVar2 == 0) {
144         puVar5 = URL_STRING_2(&local_548);
145         std::basic_string<>::assign(local_220, (basic_string<> *)puVar5, 0, 0xfffffffffffffffe);
146         if (0xf < local_528) {
147             free((void *)CONCAT71(uStack_53f, local_540));
148         }
```

Sony Felica “felica_coinst.dll (2/2)”

```
58 LAB_0200dd36:
59   if (lVar5 != -1) {
60       /* Substring after last "/" */
61       pbVar2 = FUN_02014624(param_2,local_40,lVar5 + 1,0xffffffffffffffff);
62       /* Filename NO sanitization */
63       std::basic_string<>::assign((basic_string<> *)param_1,pbVar2,0,0xffffffffffffffff);
64       if (0xf < local_20) {
65           free(local_38);
66       }
67   }
```

```
169         std::basic_string<>::assign((basic_string<> *)&local_4a8,"pUrl",4);
170         FUN_02015e10((longlong)&local_4a8,(longlong)&local_548);
171         DOWNLOAD_TO_TEMP(pcVar10,"url_list.txt");
172         local_528 = 0xf;
173         local_530 = 0;
174         local_540 = 0;
```

```
62   std::basic_string<>::assign(local_518," at Util::Download().",0x15);
63   GetTempPathA(0x104,local_148);
64   local_578 = 0xf;
65   local_580 = 0;
66   local_590 = 0;
```

SONY + SIERRA - Sony recap

4 emulate Sony FeliCa 054C:06C3

SONY

felica_coinst1050.dll ClassInstaller (DIF 0x1e) as SYSTEM pulls url_list / ins_list / app_info over plaintext HTTP from the spoofed Sony host



5 path traversal -> arbitrary write to System32

SONY

filename scan stops at / only (no .. / \ filter) -> SYSTEM writes
..\..\..\Windows\System32\WUC64.dll



6 re-emulate Sierra -> load the planted DLL

SIERRA

SmiService.exe loads WUC64.dll from System32



7 code execution as NT AUTHORITY\SYSTEM

DllMain runs as SYSTEM - pre-login console / reverse shell

NoPlug & pwn - No hardware required?

Standard-user RDP -> SYSTEM - no device/no admin/remote

- You don't have to plug something into the machine.
- RDP USB redirection: forward USB into remote session.
- The client describes the hardware. The server "builds" the device.

NoPlug & pwn - USB_REDIR umrddp.dll (1/2)

```
68         /* proceed only if fDisableUSBRedir is absent / 0 (USB redir NOT disabled) */
69     if (((LVar5 != 0) || (local_84 != 4)) || ((local_88 != 4 || (local_80[0] == 0)))) {
70         uVar12 = (uint)param_2;
71         bVar3 = FUN_18002f650(pHVar9,uVar12);
72         if ((int)CONCAT71(extraout_var,bVar3) != 0) {
73             *(undefined*)(param_1 + 0x78) = 0;
74             local_88 = 4;
75             LVar5 = RegQueryValueExW(local_78,L"MaxNumUsbDevices",(LPDWORD)0x0,&local_84,(LPBYTE)local_80,
76                                     &local_88);
```

```
95     if (((-1 < (int)uVar7) &&
96         (uVar8 = FUN_18002d9ac((uint*)(param_1 + 0x50),uVar12,(undefined4*)local_60),
97         puVar11 = local_50, (int)uVar8 != 0)) && (puVar11 = puVar10, iVar4 == 0)) {
98         /* ADD_DEVICE / announce to PnP */
99         ADD_DEVICE_FUN_18002fb94(param_1,uVar12,iVar4 == 0);
100     }
```

NoPlug & pwn - USB_REDIR termsrv.dll

```
9          /* default = E_INVALIDARG */
10  uVar3 = 0x80070057;
11  if ((param_2 != (uint *)0x0) && (param_4 != 0)) {
12      /* compare key to
          {93D359D5-831F-47B4-90BE-8383AF8F1B0E}
          */
13      lVar1 = 0x47b4831f93d359d5 - *param_1;
14      if (lVar1 == 0) {
15          lVar1 = 0xe1b8faf8383be90 - param_1[1];
16      }
```

```
36          /* BIT 11 of WinStation config = fDisablePNPRedir */
37  if (lVar1 == 0) {
38      uVar2 = *(uint *)(param_4 + 0x1f00) >> 0xb & 1;
39  }
40  else {
41      uVar2 = *(uint *)(param_4 + 0x1f00) >> 0xc & 7;
42  }
43      /* write value -> consumer reads *(p+4) */
44      *(uint *)(param_3 + 4) = uVar2;
45      uVar3 = 0;
46  }
47  }
48  return uVar3;
```

NoPlug & pwn - USB_REDIR umrdp.dll (2/2)

```
13  local_res8 = (short *)0x0;
14          /* cVar1 = ok;  DAT_180046210 = {93D359D5-...} PROPERTYKEY */
15  cVar1 = WinStationGetConnectionProperty(param_2,&DAT_180046210,&local_res8);
16          /* iVar4 = 1;!-- DEFAULT = DISABLED */
17  iVar4 = 1;
18  if ((cVar1 != '\0') && (*local_res8 == 1)) {
19          /* take the policy value (fDisablePNPRedir) */
20      iVar4 = *(int *)(local_res8 + 4);
21  }
22  if (local_res8 != (short *)0x0) {
23      WinStationFreePropertyValue();
24  }
25  if (((((undefined **)PTR_LOOP_180051000 != &PTR_LOOP_180051000) &&
26      ((PTR_LOOP_180051000[0x1c] & 1) != 0)) && (2 < (byte)PTR_LOOP_180051000[0x19]))) {
27      uVar2 = FUN_18000daf4();
28      pwVar3 = L"Enabled";
29      if (iVar4 != 0) {
30          pwVar3 = L"Disabled";
31      }
32      FUN_180014324(*(undefined8 *) (PTR_LOOP_180051000 + 0x10),0x50,&DAT_180049058,uVar2,pwVar3);
33  }
34          /* ALLOW device announce ONLY when value == 0 */
35  return iVar4 == 0;
36 }
```

NoPlug & pwn - USB_REDIR RealSense POC

1 Intel RealSense F200 - DLL hijack

oem49.inf 8086:0A66

PnP install (DIF) - co-installer runs as SYSTEM

RealSenseF200Coinstaller_227975.dll (WHQL) -> launches Setup.exe [SYSTEM]



2 co-installer drops Setup.exe

copies ...\\realsensef200depth.inf_amd64_*\\Setup.exe ->
C:\\Intel\\RSDCM\\Setup.exe



3 ProcMon - insecure DLL search order in Setup.exe

PROCMON

Setup.exe CreateFile C:\\Intel\\RSDCM\\CRYPTBASE.dll NAME NOT FOUND <- we
plant it here

Setup.exe CreateFile C:\\Windows\\System32\\CRYPTBASE.dll SUCCESS (normal
fallback)



4 code execution as NT AUTHORITY\\SYSTEM

C:\\Intel is writable by normal users, and Setup.exe probes its own dir
first -> our CRYPTBASE.dll loads, DllMain runs as SYSTEM

Also confirmed hijackable: profapi, IPHLPAPI, WINHTTP, ... (12 candidates)

POC_1 - USB_REDIREL LPE

Same Path, Third Road

Beyond the last POC - PnP becomes the loader

- The last POC was just one example.
- The real target was the install path Windows PnP fetches & loads hundreds of vulnerable signed packages straight from Windows Update. Fully trusted.
- No CreateService nor admin. The classic precondition evaporates.
- Two roads, same path: USB cable + FaceDancer (physical) and a standard RDP session (remote). We also have a third road.

PnP as the Loader

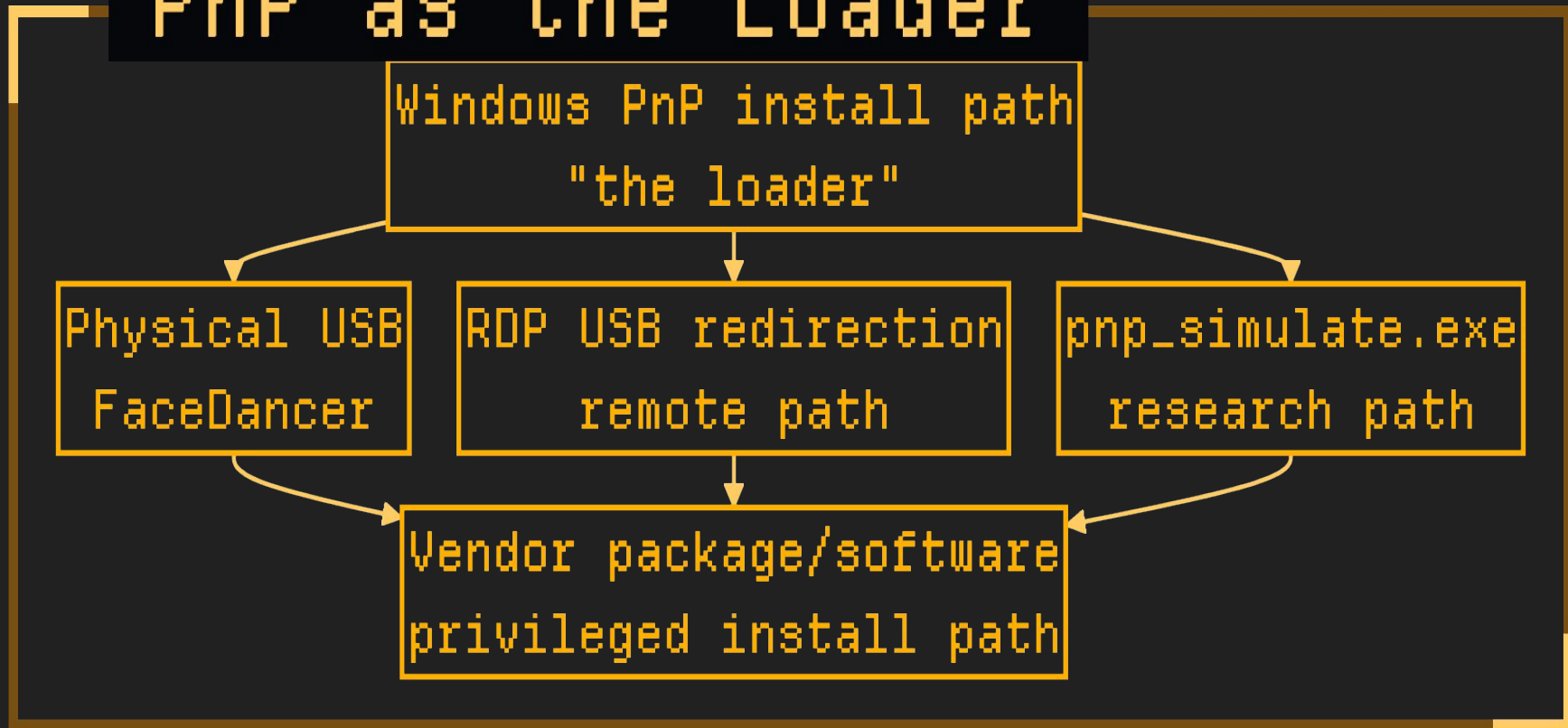
Windows PnP install path
"the loader"

Physical USB
FaceDancer

RDP USB redirection
remote path

pnp_simulate.exe
research path

Vendor package/software
privileged install path



From Features to Primitives

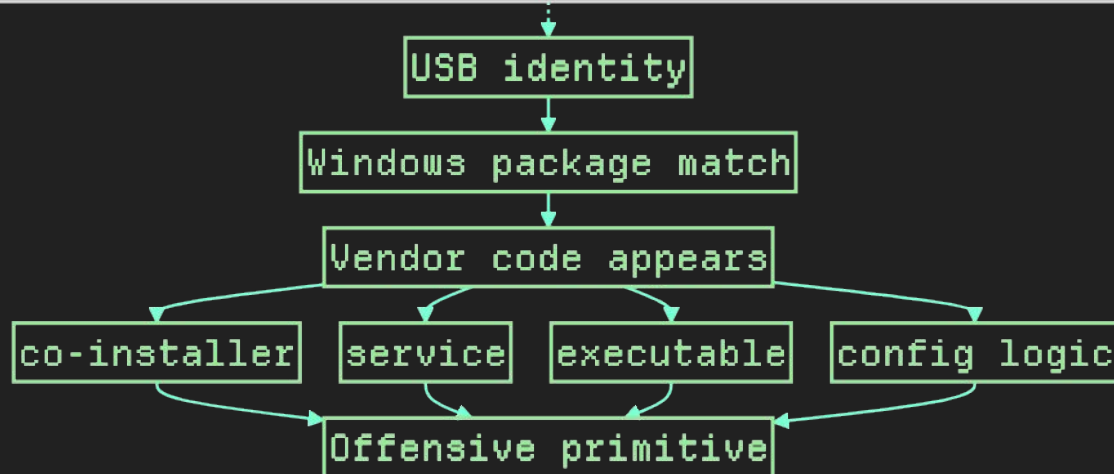
```
/*
```

```
    We were not hunting a single vendor bug.
```

```
    We mapped packages Windows can bind to USB identities.
```

```
    The real surface starts when vendor code enters the install path.
```

```
*/
```



From packages to USB identities

1 collect packages

WU/CATALOG

Microsoft Catalog + Windows Update -> CABs -> INFs -> Hardware IDs



2 triage vendor code

PE

co-installers, services, standalone EXEs, imports, registry writes, named pipes, service controls



3 reproduce the identity

USB/PNP

FaceDancer backends: Cynthion + GreatFET; plus pnp_simulate.exe for controlled Windows-side testing



/* no real vendor hardware required

COMMENT

Windows only sees an identity: VID, PID, class, interfaces and descriptors.
The rest is the normal PnP path. */

pnp_simulate.exe

1 run pnp_simulate.exe

ADMIN

--vid XXXX --pid XXXX; builds USB\VID_XXXX&PID_XXXX; starts observer via CM_Register_Notification.



2 SetupDi device creation

DEVCON

SetupDiCreateDeviceInfoList;
SetupDiCreateDeviceInfoW(ROOT, DICD_GENERATE_ID); set hardware ID.



3 DIF_REGISTERDEVICE -> device tree

KEY STEP

Device visible to PnP Manager, Device Manager and Windows Update; creates ROOT\PNPSIMDEVICE\0000.

pnp_simulate.exe

4 DiInstallDevice -> DriverStore

QUERY

Local driver search only; local INFs; writes setupapi.dev.log.



5 WU COM API -> catalog search

QUERY

IUpdateSearcher::Search; catalog metadata for discovery; not proof of auto-install.



6 --install -> CM_Setup_DevNode(READY)

INSTALL

Real install path; same API boundary into kernel. Without --install: query-only, no system changes.

Catalog Hit != Auto-Install

WU COM API / IUpdateSearcher

DISCOVERY

- > Returns catalog metadata.
- > Great for finding candidates.
- > Includes automatic, manual, old, and catalog-only packages.

```
/* A Windows Update hit is not proof that PnP will auto-  
download or install that package. */
```

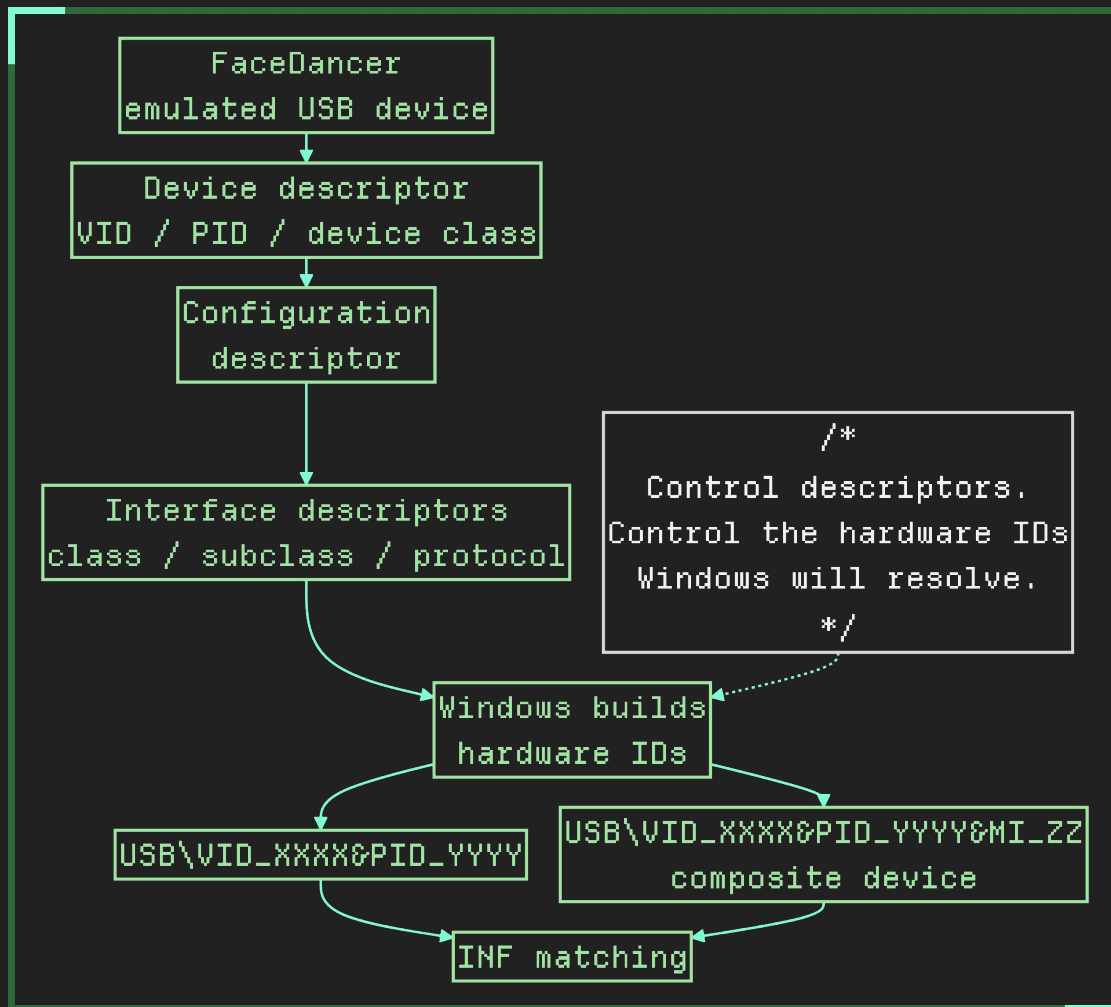
!=

Device Installation Service / Server-Side Resolve

AUTO-INSTALL PATH

- > Triggered by CM_Setup_DevNode(READY).
- > Runs through DsmSvc as a privileged install flow.
- > Restricted path: far fewer candidates.
- > Match -> CAB staging -> drvinst.exe install path.

Phase 1: Device Identity



Phase 2: USB identity to devnode

1 real USB path

HARDWARE

USB hub reads descriptors from the emulated device.



2 PDO

KERNEL

Creates a Physical Device Object for the child device.



3 BusRelations

PNP

IoInvalidateDeviceRelations(BusRelations) tells PnP: new child device.

Phase 2: USB identity to devnode

4 pnp_simulate.exe path

USER MODE

No full USB bus emulation. We reproduce the useful part from user-mode.



5 Hardware IDs

SETUPDI

SetupDiSetDeviceRegistryProperty(SPDRP_HARDWAREID).



6 Register devnode

DEVCON

SetupDiCallClassInstaller(DIF_REGISTERDEVICE).



7 Same research target

COMMENT

Both paths give us a devnode with controlled hardware IDs that Windows tries to resolve.

Phase 3: CM_Setup_DevNode

1 CM_Setup_DevNode(READY)

WINDBG

CM_Setup_DevNode(devInst, CM_SETUP_DEVNODE_READY) enters
cfgmgr32!Local_CM_Setup_DevNode.



2 DeviceIoControl(0x47084F)

KERNEL BOUNDARY

The request crosses into the Configuration Manager / PnP kernel path.



3 PiCM* handling

KERNEL

PiCMFastIoDeviceDispatch -> PiCMHandleIoctl -> PiCMDeviceAction.



4 PiQueueDeviceRequest

ASync

The action is queued. User-mode returns, but the real PnP work continues asynchronously.

Phase 4: Device Installation Service

1 PnP worker

KERNEL

Queued action picked up by the PnP worker; device IDs and compatibility are resolved.



2 Device Installation Service

SYSTEM

DsmSvc runs inside svchost.exe as SYSTEM and handles the privileged install flow.



3 Package resolution

FILTERED

Local DriverStore first. If needed: Server-Side Resolve -> Windows Update CAB staging.



4 drvinst.exe install path

VENDOR CODE

Co-installers, support executables or services enter the privileged installation path.

POC_2: Internals Demo

Signed does not mean safe logic

Cryptographic trust

SIGNED PACKAGE

- > Valid catalog / known package.
- > Windows can resolve and stage it.
- > PnP can enter the privileged install path.

```
/* Origin and install trust. Not a vendor  
logic audit. */
```

! =

Vendor logic

ATTACK SURFACE

- > Co-installers, services, support EXEs.
- > Registry values, debug flags, config paths.
- > Privileged code accepting controlled state.
- > Impact appears when pieces compose.

```
/* No UAC prompt does not mean no privileged  
action. */
```

Wacom: The Bugdoor

```
050      /* WTabletServiceISD.exe
      SHA256 955fb51d0fc2b3ccb192b6ed707771a99d28be2e7c1a7541be7762207325c86f
      Signed vendor service. The interesting part is privileged logic,
      not memory corruption. */

071      RegOpenKeyExA(HKLM,
072                  "SYSTEM\\CurrentControlSet\\Services\\WTabletServiceISD\\Service",
073                  ...);

080      /* Registry trigger verified at 0x140036950 -> 0x140036981 */
081      RegQueryValueExA(hKey, "PowerT", NULL, &type, (BYTE *)&value, &cbData);
082
083      if (type == REG_BINARY && cbData == 4 &&
084          value == 0x346b4c7f) {      /* bytes in registry: 7F 4C 6B 34 */
085          g_PowerT_enabled = true;    /* sete byte [0x140093be2] */
086      }

120      /* Later: execution path controlled by that flag */
121      if (g_PowerT_enabled) {      /* checked around 0x14002f18f */
122          startup.lpDesktop = L"WinSta0\\default";    /* 0x14002f1b2 */
123          cmdline           = L"cmd.exe";            /* 0x14002f226 */
124
125          /* Token/session plumbing is present in the binary:
126             WTSQueryUserToken / OpenProcessToken / DuplicateTokenEx */
127          CreateProcessAsUserW(token, NULL, cmdline, ...);
128      }
```

Wacom Trigger Conditions

1 registry key

HKLM

Service key: WTabletServiceISD\\Service



2 trigger value

PowerT

REG_BINARY with bytes 7F 4C 6B 34



3 binary checks

VERIFIED

type == 3, cbData == 4, compare with 0x346b4c7f



4 service context

SYSTEM

WTabletServiceISD.exe runs as LocalSystem.



5 process path

INTERACTIVE

CreateProcessAsUserW -> WinSta0\\default -> cmd.exe



6 demo boundary

COMMENT

Standalone demo writes HKLM as admin only to isolate Wacom behavior.

POC_3: Wacom

Atheros

`AdminService.exe`

```
/* Found in our pipeline; aligns with public  
CVE-2019-10617 */
```

```
// AtherosSvc / AdminService.exe  
// service context: LocalSystem
```

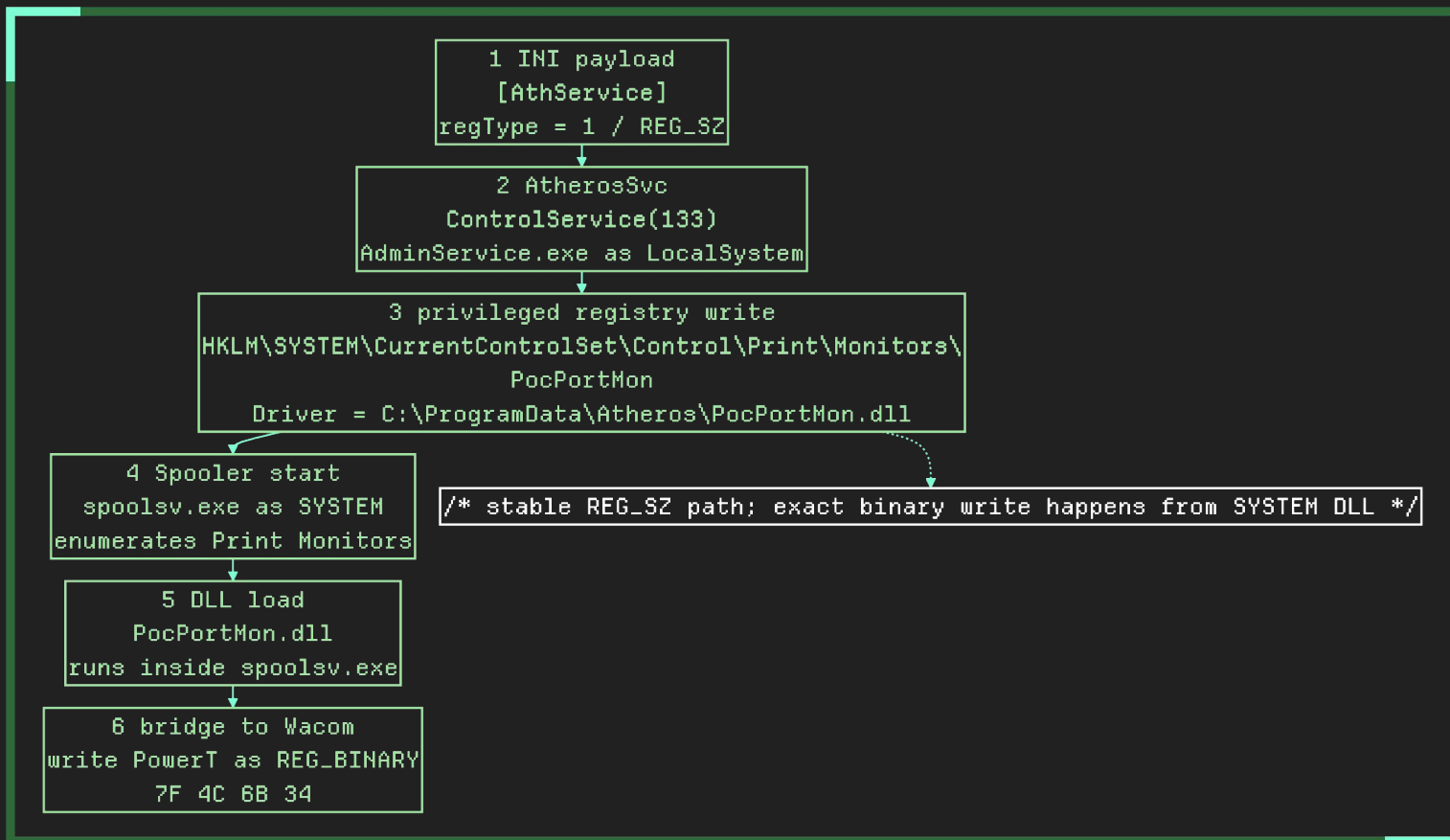
`C:\ProgramData\Atheros\AtherosServiceConfig.ini`

```
[AthService]  
regOpType="3"           // write value  
regPath="HKEY_LOCAL_MACHINE\...\Print\Monitors\PocPortMon"  
regValue="Driver"  
regType="1"             // REG_SZ  
regData="C:\ProgramData\Atheros\PocPortMon.dll"
```

```
ControlService(AtherosSvc, 133)  
    -> GetPrivateProfileStringW("AthService", "reg*", ...)  
    -> RegCreateKeyExW / RegOpenKeyExW  
    -> RegSetValueExW(...)
```

```
/* user controls the INI; the service provides the  
privilege */  
/* result: HKLM registry write as SYSTEM */
```


Atheros INI -> Registry



The chain we wanted vs the chain that worked

Direct Wacom chain

TOO CLEAN

- > Atheros writes PowerT as REG_BINARY.
- > Restart WTabletServiceISD.
- > Wacom launches cmd.exe as SYSTEM.
- > Problem: AdminService.exe parses binary data character-by-character.
- > "7F4C6B34" becomes the wrong byte sequence.

```
/* Pretty chain. Wrong parser behavior for  
this target. */
```

!=

Working chain

STABLE PATH

- > Atheros writes a Print Monitor REG_SZ.
- > spoolsv.exe loads PocPortMon.dll as SYSTEM.
- > The DLL writes the exact Wacom PowerT value.
- > RegSetValueExW: 7F 4C 6B 34.
- > Restart Wacom -> interactive SYSTEM shell.

```
/* Registry write -> SYSTEM DLL -> exact  
Wacom trigger. */
```

POC_4: Full Chain From Low Priv

Mitigations

- Physically rip out the USB ports
 - Fill the ports with silicone or cement
 - Cut the power cable to the user's machine
-
- Restrict package delivery via Windows Update
 - Allow-lists by hardware ID / device class
 - Block unnecessary USB classes
 - Audit vendor services that persist running as SYSTEM
 - Disable RDP USB redirection in remote environments



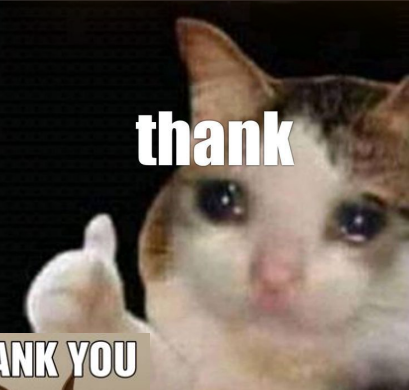
Conclusions

- A physical USB identity can lead to SYSTEM execution with 0 clicks
- In certain environments, a USB identity redirected over RDP can result in privilege escalation
- A bug that looks like it requires admin can chain into a full LPE on the latest Windows with a second bug that has been there since 2019
- Windows can pull vendor software on our behalf with system permissions just by inserting a USB
- If that software exposes the right primitive, PnP stops being just device installation, It becomes an execution path

References

- CVE Program. (s. f.). CVE-2019-10617. <https://www.cve.org/CVERecord?id=CVE-2019-10617>
- National Institute of Standards and Technology. (s. f.). CVE-2019-10617 detail. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2019-10617>
- Qualcomm Technologies, Inc. (2019, october). Qualcomm Security Bulletin: October 2019. <https://docs.qualcomm.com/product/publicresources/securitybulletin/october-2019-bulletin.html>
- Microsoft. (s. f.). Hardware IDs. Microsoft Learn. <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/hardware-ids>
- Microsoft. (s. f.). SetupDiCreateDeviceInfoA function. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/setupapi/nf-setupapi-setupdicreatedeviceinfoa>
- Microsoft. (s. f.). DIF_REGISTERDEVICE installation request. Microsoft Learn. <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/dif-registerdevice>
- Microsoft. (s. f.). CM_Setup_DevNode function. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/cfgmgr32/nf-cfgmgr32-cm_setup_devnode
- Microsoft. (s. f.). DiInstallDevice function. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/newdev/nf-newdev-diinstalldevice>
- Microsoft. (s. f.). Driver Store. Microsoft Learn. <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/driver-store>
- Microsoft. (s. f.). IUpdateSearcher interface. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/wuapi/nn-wuapi-iupdatesearcher>
- Yosifovich, P., Russinovich, M. E., Ionescu, A., & Solomon, D. A. (2017). Windows Internals, Part 1: System architecture, processes, threads, memory management, and more (7th ed.). Microsoft Press. <https://www.microsoftpressstore.com/store/windows-internals-part-1-system-architecture-processes-9780735684188>
- Allievi, A., Russinovich, M. E., Ionescu, A., & Solomon, D. A. (2021). Windows Internals, Part 2 (7th ed.). Microsoft Press. <https://www.microsoftpressstore.com/store/windows-internals-part-2-9780135462409>
- Great Scott Gadgets. (s. f.). FaceDancer [GitHub repository]. GitHub. <https://github.com/greatscottgadgets/facedancer>
- Great Scott Gadgets. (s. f.). Cynthion. <https://greatscottgadgets.com/cynthion/>
- Great Scott Gadgets. (s. f.). Cynthion [GitHub repository]. GitHub. <https://github.com/greatscottgadgets/cynthion>
- Great Scott Gadgets. (s. f.). GreatFET [GitHub repository]. GitHub. <https://github.com/greatscottgadgets/greatfet>

plugandpwn.com



thank

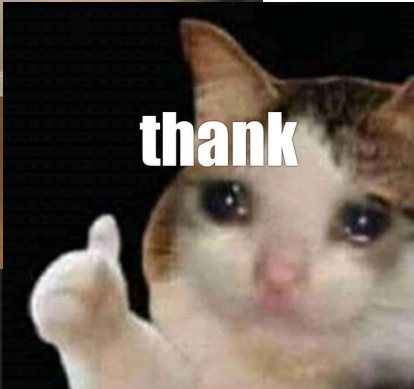


thank



THANK YOU

SO MUCH



thank



THANKS



thank



thank



THANK YOU

SO MUCH